

# Model Binding Using FromBody in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

## Model Binding Using FromBody in ASP.NET Core Web API

In this article, I will discuss How to Implement **Model Binding Using FromBody in ASP.NET Core Web API** Application with Examples. Please read our previous article discussing Model Binding Using [FromBody in ASP.NET Core Web API](#) with Examples.

### What is FromBody Attribute in ASP.NET Core Web API?

The FromBody attribute in ASP.NET Core Web API is used to indicate that an action method parameter should be bound from the body of the incoming HTTP request. When a parameter is decorated with the FromBody attribute, ASP.NET Core will attempt to deserialize the body of the request (typically in JSON format) into the type specified by the parameter. This is typically used for parameters that are complex types or data objects sent as JSON or XML formats in the body of the HTTP request. If you go to the definition of FromBodyAttribute, you will see the following signature.

```
public class FromBodyAttribute : Attribute, IBindingSourceMetadata, IConfigureEmptyBodyBehavior
{
    public BindingSource BindingSource => BindingSource.Body;

    public EmptyBodyBehavior EmptyBodyBehavior { get; }
    bool IFromBodyMetadata.AllowEmpty => EmptyBodyBehavior == EmptyBodyBehavior.Allow;
}
```

The FromBodyAttribute specifies that a parameter or property should be bound using the request body.

### How Does the FromBody Attribute Work in ASP.NET Core Web API?

When the FromBody attribute is applied to a parameter, ASP.NET Core uses configured formatters based on the content type of the request (e.g., application/json, application/xml) to deserialize the request body into a C# object. The model binding system handles this process, choosing the appropriate formatter to parse the request body and map it to the defined parameter based on the content type.

```
[HttpPost]
public IActionResult CreateUser([FromBody] User user)
{
    // The 'user' parameter is populated from the JSON body of the request
    // Process the 'user' object here
}
```

```
    return Ok();  
}
```

In this example, if a POST request is sent with a JSON body like { "Name": "John", "Age": 30 }, the user parameter will be populated with a User object with Name set to "John" and Age set to 30.

## Example to Understand FromBody Attribute in ASP.NET Core Web API

Let's start with a basic example: We have a simple model class and an action method in a controller that uses [FromBody].

### Creating Model:

First, create a class file named **Product.cs**, and then copy and paste the following code. This is the model class that we will use as a parameter in our action method.

```
namespace ModelBinding.Models  
{  
    public class Product  
    {  
        public int Id { get; set; }  
        public string Name { get; set; }  
        public int Quantity { get; set; }  
        public int Price { get; set; }  
    }  
}
```

### Creating Controller:

Next, create an API Controller named **ProductsController** and copy and paste the following code.

```
using Microsoft.AspNetCore.Mvc;  
using ModelBinding.Models;  
  
namespace ModelBinding.Controllers  
{  
    [Route("api/[controller]")]  
    [ApiController]  
    public class ProductsController : ControllerBase  
    {  
        [HttpPost]  
        public IActionResult CreateProduct([FromBody] Product product)  
        {  
            // Add the product to the database or in-memory store  
            // For demonstration, let's return the product back  
        }  
    }  
}
```

```
        return Ok(product);  
    }  
}  
}
```

In this example, when you send a POST request with a JSON body representing a Product, the ASP.NET Core model binding system will automatically populate the product parameter with the data from the request body. To test the above API, please use the following details:

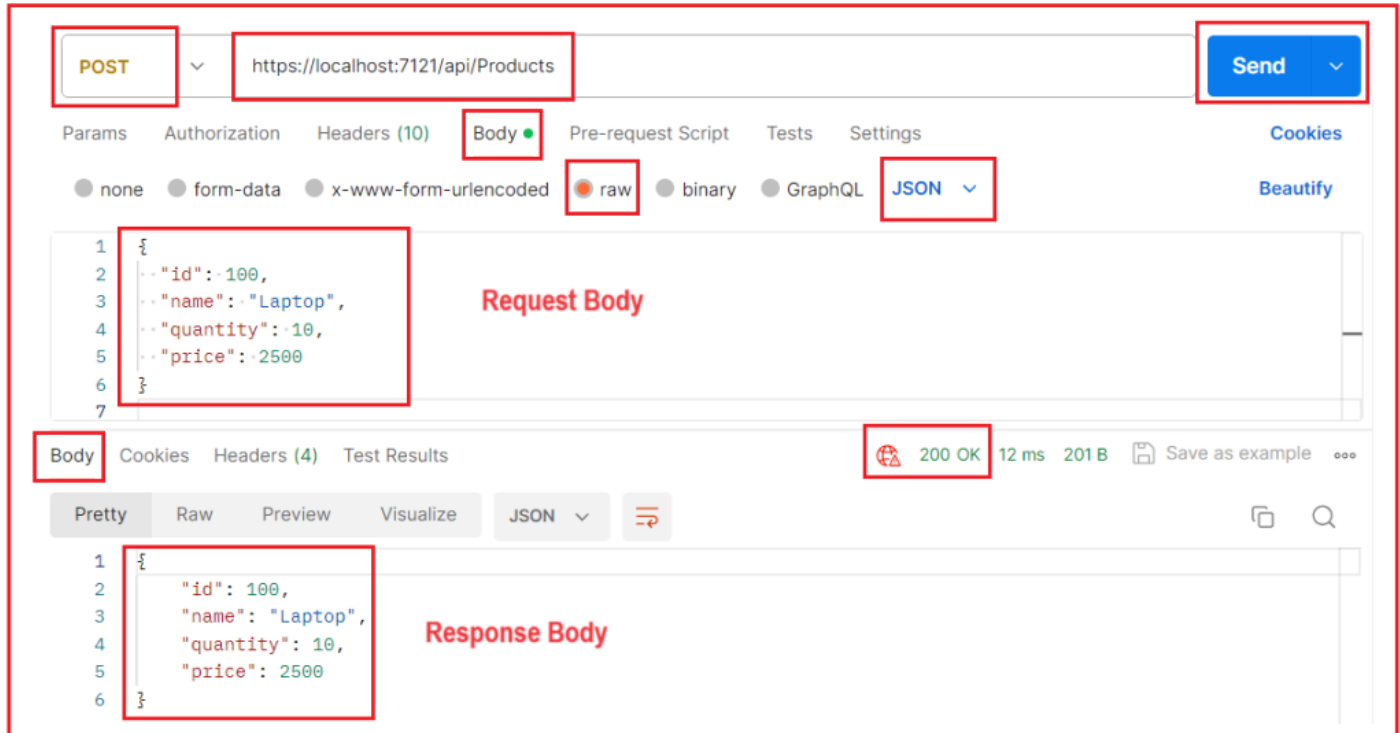
**Method: POST**

**URL: api/products**

**Request Body:**

```
{  
  "id": 100,  
  "name": "Laptop",  
  "quantity": 10,  
  "price": 2500  
}
```

You can test it using Postman, as shown in the image below. Here, you need to change the API endpoint port number where your application is running.



## Handling Collection or Arrays using FromBody in ASP.NET Core Web API

Model binding can also handle arrays or collections. This is useful when you want to send a list of items in a single request. For a better understanding, please modify the Products Controller as follows:

```
using Microsoft.AspNetCore.Mvc;
using ModelBinding.Models;

namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
    {
        [HttpPost]
        public IActionResult CreateProduct([FromBody] List<Product> products)
        {
            // Handle multiple products
            // For demonstration, let's return the products back
            return Ok(products);
        }
    }
}
```

In this example, you can POST an array of products in JSON format, and it will be bound to the products parameter as a collection of Product objects. To test the above API, please use the following details:

**Method: POST**

**URL: api/products**

**Request Body:**

```
[
  {
    "id": 100,
    "name": "Laptop",
    "quantity": 10,
    "price": 2500
  },
  {
    "id": 101,
    "name": "Desktop",
    "quantity": 15,
    "price": 2900
  }
]
```

## More Complex Model Binding:

Assume a more complex model that includes nested objects or collections. So, modify the **Products.cs** class file as follows:

```
namespace ModelBinding.Models
{
    public class Order
    {
        public int Id { get; set; }
        public List<Product> Products { get; set; }
    }

    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Quantity { get; set; }
        public int Price { get; set; }
    }
}
```

Next, modify the **Products Controller** as follows:

```
using Microsoft.AspNetCore.Mvc;
using ModelBinding.Models;

namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
    {
        [HttpPost("Order")]
        public IActionResult CreateOrder([FromBody] Order order)
        {
            // Process the order
            return Ok(order);
        }
    }
}
```

The above example allows for a complex JSON object representing an order with an array of products to be posted and bound to the order parameter. To test the above API, please use the following details:

**Method: POST**

**URL: api/Products/Order**

**Request Body:**

```
{
  "id": 12345,
  "products": [
    {
      "id": 1,
      "name": "Laptop",
      "quantity": 1,
      "price": 2500
    },
    {
      "id": 2,
      "name": "Desktop",
      "quantity": 2,
      "price": 5000
    }
  ]
}
```

## Customizing Serialization and Deserialization in ASP.NET Core Web API

You can customize the deserialization process by configuring the options for the input formatters in the Program.cs class file. By default, ASP.NET Core Framework uses CamelCase naming conventions. This means that the first letter of the identifier is lowercase, and the first letter of each subsequent concatenated word is uppercase. If you want to ignore this naming convention, then you need to configure the same serialization settings using `AddJsonOptions` as follows:

```
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.PropertyNamingPolicy = null;
    });
```

With the above changes in place, run the application, and it should work with the original property names. You can test the same as follows:

**Method: POST**

**URL: `api/Products/Order`**

**Request Body:**

```
{
  "Id": 12345,
  "Products": [
    {
      "Id": 1,
      "Name": "Laptop",

```

```

        "Quantity": 1,
        "Price": 2500
    },
    {
        "Id": 2,
        "Name": "Desktop",
        "Quantity": 2,
        "Price": 5000
    }
]
}

```

## Response Body:

```

1  {
2      "Id": 12345,
3      "Products": [
4          {
5              "Id": 1,
6              "Name": "Laptop",
7              "Quantity": 1,
8              "Price": 2500
9          },
10         {
11             "Id": 2,
12             "Name": "Desktop",
13             "Quantity": 2,
14             "Price": 5000
15         }
16     ]
17 }

```

## What are CamelCase and PascalCase?

In Programming, camelCase and PascalCase are two conventions for naming identifiers such as variables, classes, methods, and other entities in code. These conventions indicate how multi-word identifiers are joined together. Despite their similarities, they differ primarily in the capitalization of the first letter.

### CamelCase

In CamelCase, the first letter of the identifier is lowercase, and the first letter of each subsequent concatenated word is uppercase. It is also known as lowerCamelCase. Commonly used for naming variables, methods, or functions in many programming languages. For Example: myVariableName, calculateTotalWidth, readUserData

## PascalCase

In PascalCase, the first letter of each concatenated word, including the first word, is uppercase. This makes all the word boundaries distinct with capital letters. It is sometimes referred to as UpperCamelCase. It is often used for naming classes, interfaces, namespaces, and other constructs that define types or significant structures in code. For Example: MyVariableName, CalculateTotalWidth, ReadUserData

## When Should We Use FromBody Attribute in ASP.NET Core Web API?

We need to use the FromBody attribute when we need to bind a complex object (e.g., a class with multiple properties) from the raw data in the HTTP request body. This is commonly used in POST, PUT, and PATCH requests, where the client sends data to the server in the body of the request, usually in JSON format, and you want to map this data directly to a model. So, we need to use the FromBody attribute when:

- The method expects a complex object that comes from the client in the request body.
- You are working with POST or PUT HTTP methods where the request body contains a serialized object that needs to be deserialized into a model.
- The data you need cannot be easily represented or appended in the URL (e.g., a complex object or a large amount of data).

**Note:** The FromBody attribute does not have a Name property. The Name property is associated with attributes like FromQuery, FromRoute, and FromHeader, where it specifies the name of the query string parameter, route parameter, or header key.

In the next article, I will discuss How to Implement [Custom Model Binding in ASP.NET Core Web API](#) with Examples. In this article, I try to explain **Model Binding Using FromBody in ASP.NET Core Web API** with Examples. I hope you enjoy this article, “Model Binding Using FromBody in ASP.NET Core Web API.”



Dot Net Tutorials

**About the Author: Pranaya Rout**

*Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.*



## 2 thoughts on “Model Binding Using FromBody in ASP.NET Core Web API”



**WALKER ZHANG**

APRIL 19, 2025 AT 4:48 PM

ASP.NET Core 6 Web API : ModelBinder failed to convert time string into datetime

[HttpPost]

```
public IActionResult methodOne([FromForm] DtoWithDateTime dtoWithDateTime )
```

[HttpPost]

```
public IActionResult methodTwo([FromBody] DtoWithDateTime dtoWithDateTime )
```

If I use dtoWithDateTime with time property “2025-04-16 16:21:33” to call the Web API methods, the methodOne works fine, but methodTwo failed. I understand why methodTwo failed due to time format here.

My question is why the same parameter from request works fine with methodOne which uses FromForm attribute?

Thanks,

Walker

[Reply](#)



**DANNIE MYERS**

DECEMBER 22, 2025 AT 5:30 AM

This tutorial is excellent!!

[Reply](#)

### Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information